
Bulla Factoring V2.2 Audit Report

Prepared by [Oxleastwood](#) & [kamensec](#)

Version 1.0

Lead Auditors

[Oxleastwood](#)

[kamensec](#)

July 30, 2026

Contents

1	About LPHAC	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
7	Findings	5
7.1	Critical	5
7.1.1	Recovered impaired invoices can be reconciled repeatedly, inflating LP gains	5
7.2	High	6
7.2.1	Impaired-invoice recovery double-counts the recovered amount, leaving the pool insolvent	6
7.2.2	processRedemptionQueue underflows on a fully-draining asset withdrawal, freezing deposits/reconciles/impairments	9
7.3	Medium	12
7.3.1	unfactorInvoice does not reclaim insurancePremium causing insurance balance solvency deficit	12
7.3.2	deposit only validates the caller, so compliant users can mint pool shares to non-compliant recipients	13
7.3.3	Factory-created pools permanently lose RedemptionQueue admin control	13
7.3.4	Impairment principalLoss clamp discards positive surplus, permanently stranding pool funds	14
7.3.5	Partial payments on an impaired invoice are never credited and become permanently stranded	16
7.4	Low	19
7.4.1	impairmentGrossGainBps is read live in previewImpair, allowing retroactive change of insurance economics for already-funded invoices	19
7.4.2	Impairment released the withheld-fee wedge without writing it down, leaving the pool insolvent by fundedAmountGross - fundedAmountNet	20
7.4.3	Funding an invoice with upfrontBps < protocolFeeBps + insuranceFeeBps makes the pool instantly insolvent	21
7.5	Informational	23
7.5.1	viewPoolStatus pagination assumes stable iteration order which EnumerableSet does not provide	23
7.5.2	Queued redemptions bypass current redeemPermissions after permissions are revoked	23
7.5.3	createPool emits and accepts permission/name metadata that it never verifies against the deployed pool	24

1 About LPHAC

LPHAC is a Web3 security organization aiming to protect projects and their partners against malicious actors. We aim to provide holistic improvements to a project's security stack in a safe and reliable way.

2 Disclaimer

The LPHAC team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Bulla Network enables the on-chain creation of credit pools for invoice factoring. By adhering to the **ERC4626** specification, permissioned depositors can earn interest for facilitating invoice funding. Through these contracts, invoice issuers can factor their receivables, allowing them to receive early payments in exchange for a premium. This integration not only broadens the utility of the Bulla Claim Protocol but also provides a new financial mechanism for liquidity and credit management on-chain.

Version 2.2 of the factoring contracts introduces batched invoice funding for improved throughput, removes direct loan offers from the pool, and adds an insurer-driven impairment workflow that allows funded losses to be absorbed by a designated insurer in exchange for a configurable premium and recovery share.

5 Audit Scope

The audit was conducted against initial commit [20ec4a6](#) over **3** days. A subsequent fix review was conducted against commit [eb28854](#), with the resulting fixes verified at final commit [5c67916](#). The finalized audit scope is documented at [V2_2_AUDIT_SCOPE.md](#).

The following contracts were included as part of the review scope, including any related external dependencies:

```
contracts
  BullaFactoring.sol
  BullaClaimV2InvoiceProviderAdapterV2.sol
  ComplianceDepositPermissions.sol
  libraries/FeeCalculations.sol
  libraries/ApprovalPacking.sol
interfaces
  IBullaFactoring.sol
```

6 Executive Summary

Over the course of 3 days, the LPHAC team conducted an audit on the [Bulla Factoring V2.2](#) smart contracts provided by [Bulla](#). In this period, a total of 14 issues were found.

All fix-review findings were verified against the resolutions posted by the Bulla team in the public [bulla-network/factoring-contracts](#) repository. Findings marked as Verified Fix have had their resolutions independently reviewed by LPHAC. Findings marked as Acknowledged were accepted by the Bulla team without a code change and are retained for visibility.

Summary

Project Name	Bulla Factoring V2.2
Repository	factoring-contracts
Initial Commit	20ec4a6...
Final Commit	5c67916...
Audit Timeline	May 18th - May 20th
Methods	Manual Review

Issues Found

Critical Risk	1
High Risk	2
Medium Risk	5
Low Risk	3
Informational	3
Gas Optimizations	0
Total Issues	14

Summary of Findings

[C-1] Recovered impaired invoices can be reconciled repeatedly, inflating LP gains	✓ Verified Fix
[H-1] Impaired-invoice recovery double-counts the recovered amount, leaving the pool insolvent	✓ Verified Fix
[H-2] <code>processRedemptionQueue</code> underflows on a fully-draining asset withdrawal, freezing deposits/reconciles/impairments	✓ Verified Fix
[M-1] <code>unfactorInvoice</code> does not reclaim <code>insurancePremium</code> causing insurance balance solvency deficit	✓ Verified Fix
[M-2] <code>deposit</code> only validates the caller, so compliant users can mint pool shares to non-compliant recipients	✓ Verified Fix

[M-3] Factory-created pools permanently lose <code>RedemptionQueue</code> admin control	✓ Acknowledged
[M-4] Impairment <code>principalLoss</code> clamp discards positive surplus, permanently stranding pool funds	✓ Verified Fix
[M-5] Partial payments on an impaired invoice are never credited and become permanently stranded	✓ Acknowledged
[L-1] <code>impairmentGrossGainBps</code> is read live in <code>previewImpair</code> , allowing retroactive change of insurance economics for already-funded invoices	✓ Verified Fix
[L-2] Impairment released the withheld-fee wedge without writing it down, leaving the pool insolvent by <code>fundedAmountGross - fundedAmountNet</code>	✓ Verified Fix
[L-3] Funding an invoice with <code>upfrontBps < protocolFeeBps + insurance-FeeBps</code> makes the pool instantly insolvent	✓ Verified Fix
[I-1] <code>viewPoolStatus</code> pagination assumes stable iteration order which <code>EnumerableSet</code> does not provide	✓ Verified Fix
[I-2] Queued redemptions bypass current <code>redeemPermissions</code> after permissions are revoked	✓ Verified Fix
[I-3] <code>createPool</code> emits and accepts permission/name metadata that it never verifies against the deployed pool	✓ Acknowledged

7 Findings

7.1 Critical

7.1.1 Recovered impaired invoices can be reconciled repeatedly, inflating LP gains

Context: [BullaFactoring.sol#L599-L632](#), [BullaFactoring.sol#L952-L989](#)

Description: `impairInvoice()` marks `impairmentInfo[invoiceId].isImpaired = true`, but the impaired branch of `reconcileSingleInvoice()` never clears that flag, never deletes `impairmentInfo[invoiceId]`, and never otherwise records that the recovery was already processed. Because the invoice was already removed from `_activeInvoices` during impairment, the normal `removeActivePaidInvoice()` one-shot guard used by the non-impaired path does not exist here.

Once an impaired invoice becomes fully paid, any caller can execute the impaired branch of `reconcileSingleInvoice()` over and over again. Each replay reuses the same `purchasePrice` and `paidAmountAtImpairment`, then credits the same `insuranceShare` into `insuranceBalance` and the same `investorShare` into `paidInvoicesGain` again. If `principalLoss` was zero at impairment, these replays are unbounded. If `principalLoss` was non-zero, the replay still double-counts gains until `impairmentLosses` underflows.

This creates phantom assets in internal accounting:

- the insurer can withdraw an inflated `insuranceBalance` from real pool cash via `withdrawInsuranceBalance()`;
- LP accounting is inflated through `paidInvoicesGain`, which raises `pricePerShare()` and lets an attacker who holds shares redeem more than their fair share of the pool.

Proof of concept scenario:

1. An attacker funds an invoice, or waits for a legitimate invoice, where `paymentsSinceFunding + lpCredit >= fundedAmountNet` at impairment time, so `principalLoss == 0`.
2. The insurer impairs the invoice.
3. The debtor later fully repays the invoice.
4. The first call to `reconcileSingleInvoice(invoiceId)` correctly credits recovery proceeds.
5. The attacker calls `reconcileSingleInvoice(invoiceId)` again.
6. Because `impairmentInfo[invoiceId].isImpaired` is still true, the contract re-credits the same `insuranceShare` and `investorShare`.
7. Steps 5-6 can be repeated indefinitely when `principalLoss == 0`, manufacturing arbitrary insurer balance and LP gains from a single recovered invoice.

Recommendation: Make impaired recovery one-shot. Clear or delete `impairmentInfo[invoiceId]` before any external effects are observable, or add an explicit `isRecovered` flag and revert on subsequent calls. The contract should also ensure the invoice is removed from `impairedInvoices` exactly once and cannot pass the impaired branch again after recovery has been processed.

Bulla: Resolved in [PR 235](#).

LPHAC: Verified fix. `delete impairmentInfo[invoiceId]` is now executed in the recovery branch of `reconcileSingleInvoice` immediately after the insurance share and LP gain are credited, so any subsequent call cannot re-enter the impaired branch for the same invoice.

7.2 High

7.2.1 Impaired-invoice recovery double-counts the recovered amount, leaving the pool insolvent

Context: [BullaFactoring.sol#L611-L667](#), [BullaFactoring.sol#L617-L647](#), [BullaFactoring.sol#L968-L1006](#)

Description: When the insurer impairs an invoice, the insurance payout (`impairmentGrossGain`, the insurer's `purchasePrice`) reduces the LP loss, and at that moment it is correctly backed by cash. When the debtor later repays that impaired invoice, the recovery branch pays the same payout back out to the insurer and *simultaneously* reverses the LP loss in full. The payout is credited twice, and the pool is left insolvent.

At impairment, the payout reduces the LP loss. The LP capital account is:

```
capitalAccount = totalDeposits + paidInvoicesGain - totalWithdrawals - impairmentLosses
```

At impairment the only term that moves is `impairmentLosses += _principalLoss`, so the capital account drops by `principalLoss`. Because

```
principalLoss = fundedAmountNet - lpCredit - paymentsSinceFunding
```

and `lpCredit` contains `impairmentGrossGain`, a larger payout means a smaller `principalLoss`, and therefore a smaller drop in LP capital. This is correct and fully backed here, because the pool actually receives `impairmentGrossGain` in cash at impairment (partly from `insuranceBalance`, partly from the insurer's `outOfPocketCost` transfer).

The payout is then credited a second time at recovery. When the debtor repays, `reconcileSingleInvoice` enters the [impaired branch](#):

```
// BullaFactoring.sol:625-632
uint256 insuranceShare = recoveredAmount - investorShare;

insuranceBalance += insuranceShare;           // to insurer
paidInvoicesGain += investorShare;           // to LPs
impairmentLosses -= _impairment.principalLoss; // to LPs, a second time
```

Two credits come out of one recovered amount:

1. The entire `recoveredAmount` is split between insurer and LPs (`insuranceShare + investorShare == recoveredAmount` by construction, see [L625](#)). The insurer's share reclaims their `purchasePrice` (= `impairmentGrossGain`) first.
2. `impairmentLosses -= principalLoss` reverses the LP loss in full, pushing the capital account back up by `principalLoss`.

The payout the insurer reclaims in step 1 is the same money that reduced the LP loss at impairment, but that LP benefit is never clawed back: the loss is reversed for free in step 2. So the payout benefits the LPs (reduced loss, kept) and returns to the insurer (part of `insuranceShare`) at the same time. The reversal at [L632](#) creates `principalLoss` worth of LP capital with no cash behind it.

Proof of concept: Single invoice, `invoiceAmount = 100_000`, `upfrontBps = 8000`, `CommonSetup` insurance params. After impair + full recovery, with the invoice fully resolved (`capitalAtRiskPlusWithheldFees == 0`):

quantity	value
pool token balance	1,026,691
LP capital account (alice's claim)	1,047,500
insuranceBalance	52,500
adminFeeBalance	2,554
protocolFeeBalance	250

quantity	value
sum of claims	1,102,804
deficit	76,113

The test then has the insurer withdraw `insuranceBalance` (a permitted action), after which `alice's redeem(maxRedeem, alice, alice)` reverts with ERC20 insufficient balance: the last party cannot be paid. The deficit tracks the reversed `principalLoss`.

The test below extends the team's existing `CommonSetup` harness and uses only existing helpers, so it drops straight into `test/foundry/`. Full file also at `leads/poc/TestAuditImpairRecoverySolvency.t.sol`.

Run with:

```
FOUNDRY_PROFILE=test forge test --match-contract TestAuditImpairRecoverySolvency -vv
```

(The test profile is required: the default profile hits a solc "Tag too large" internal error at `optimizer_runs=200`; the test profile uses `optimizer_runs=10`.)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./CommonSetup.t.sol";

/// @notice PoC for the impaired-invoice recovery double-count (Lead 01).
/// @dev Drop into test/foundry/ and run:
///      FOUNDRY_PROFILE=test forge test --match-contract TestAuditImpairRecoverySolvency -vv
contract TestAuditImpairRecoverySolvency is CommonSetup {
    // insurer configured in CommonSetup: insuranceFeeBps=100, impairmentGrossGainBps=500,
    //   ↪ recoveryProfitRatioBps=5000
    address insurerAddr = address(0x1999);

    /// @dev Every asset the pool owes: LP capital account + the fee/insurance buckets.
    function _totalClaims() internal view returns (uint256) {
        return bullaFactoring.calculateCapitalAccount()
            + bullaFactoring.protocolFeeBalance()
            + bullaFactoring.adminFeeBalance()
            + bullaFactoring.insuranceBalance();
    }

    function testImpairThenRecoveryLeavesPoolInsolvent() public {
        // Fund the insurer so it can cover the out-of-pocket impairment cost.
        asset.mint(insurerAddr, 1_000_000);
        vm.prank(insurerAddr);
        asset.approve(address(bullaFactoring), type(uint256).max);

        // 1. Alice (LP) deposits.
        vm.prank(alice);
        bullaFactoring.deposit(1_000_000, alice);

        // 2. Fund a single invoice (creditor=bob, debtor=charlie).
        vm.prank(bob);
        uint256 invoiceId = createClaim(bob, charlie, 100_000, dueBy);
        vm.prank(underwriter);
        _approveInvoice(invoiceId, interestApr, spreadBps, upfrontBps, 0);
        vm.startPrank(bob);
        bullaClaim.approve(address(bullaFactoring), invoiceId);
        _fundInvoice(invoiceId, upfrontBps, address());
        vm.stopPrank();
    }
}
```

```

// 3. Warp past the impairment grace period and impair.
vm.warp(block.timestamp + 91 days);
vm.prank(insurerAddr);
bullaFactoring.impairInvoice(invoiceId);

// 4. Debtor later pays in full -> reconcileSingleInvoice recovery branch
// runs automatically via the paid callback.
vm.startPrank(charlie);
asset.approve(address(bullaClaim), 100_000);
bullaClaim.payClaim(invoiceId, 100_000);
vm.stopPrank();

// The invoice is fully resolved, so no capital is deployed:
assertEq(
    bullaFactoring.calculateCapitalAccount() - bullaFactoring.totalAssets(),
    0,
    "no capital should be deployed after full resolution"
);

// 5. INSOLVENCY: outstanding claims exceed the pool's token balance.
uint256 poolCash = asset.balanceOf(address(bullaFactoring));
uint256 claims = _totalClaims();
emit log_named_uint("pool token balance", poolCash);
emit log_named_uint("sum of outstanding claims", claims);
emit log_named_uint("deficit", claims - poolCash);
assertGt(claims, poolCash, "expected pool to be insolvent (claims > cash)");

// 6. Concrete impact: the insurer exits first (a permitted action), then the
// LP cannot redeem her shares because the pool no longer holds enough tokens.
vm.prank(insurerAddr);
bullaFactoring.withdrawInsuranceBalance();

uint256 shares = bullaFactoring.maxRedeem(alice);
vm.prank(alice);
vm.expectRevert(); // ERC20 insufficient balance: pool cannot pay the LP
bullaFactoring.redeem(shares, alice, alice);
}
}

```

Recommendation: The recovered cash cannot be both distributed in full and used to reverse the LP loss. Pick one:

- Ensure LPs recover their principal first: allocate recoveredAmount in priority order (reduce impairmentLosses by the recovered principal, then repay the insurer's purchasePrice, then split any leftover per recoveryProfitRatioBps), so every payout is funded from the recovery. This changes the insurer/investor split, so the existing recovery tests need their expected numbers updated.
- Or delete the impairmentLosses -= principalLoss line ([L632](#)). This keeps the current split ratio the existing tests expect, but LPs then permanently bear the principal loss even when the invoice is fully repaid.

Bulla: Resolved in [PR 242](#).

LPHAC: Verified. PR 242 removes the recovery double-count and the deficit drops from 76,113 to a small residual. That residual (the withheld-fee wedge, fundedAmountGross - fundedAmountNet) is a separate issue, fixed in [PR 244](#).

7.2.2 processRedemptionQueue underflows on a fully-draining asset withdrawal, freezing deposits/reconciles/impairments

Context: [BullaFactoring.sol#L1115-L1185](#), [BullaFactoring.sol#L1122](#), [BullaFactoring.sol#L1157](#), [BullaFactoring.sol#L1166](#)

Description: `processRedemptionQueue` computes `maxRedeemableShares` once as `convertToShares(_totalAssets)`, which rounds **down** (`Math.Rounding.Floor`). For an asset-based queued withdrawal it burns `sharesToBurn = previewWithdraw(assetsToWithdraw)`, which rounds **up** (`Math.Rounding.Ceil`), and then decrements `maxRedeemableShares -= sharesToBurn`.

Exactly where the rounding occurs: both quantities are the same conversion $\text{assets} * \text{totalSupply} / \text{capitalAccount}$, differing only in rounding mode, and they come from two different OpenZeppelin ERC4626 helpers. `convertToShares` (L1122) resolves to `_convertToShares(assets, Math.Rounding.Floor)`, so `maxRedeemableShares = floor(_totalAssets * totalSupply / capitalAccount)`. `previewWithdraw` (L1157) resolves to `_convertToShares(assets, Math.Rounding.Ceil)`, so `sharesToBurn = ceil(assetsToWithdraw * totalSupply / capitalAccount)`. The opposite rounding is deliberate in ERC4626 (a withdrawal rounds the shares-to-burn up so the withdrawer cannot extract a sliver of extra value; the neutral valuation rounds down), but the loop then subtracts the ceil value from the floor value.

For a partial fill (`assetsToWithdraw < _totalAssets`) the ceil value stays comfortably below the floor cap, so the subtraction is safe. On a full drain (`assetsToWithdraw == _totalAssets`, which happens whenever the queued amount is at least the free liquidity, because `assetsToWithdraw = min(redemption.assets, _totalAssets)` clamps to `_totalAssets`), the two are `floor(x)` and `ceil(x)` of the *same* `x`. If `x` has any remainder, `ceil(x) = floor(x) + 1`, so `sharesToBurn = maxRedeemableShares + 1` and `maxRedeemableShares -= sharesToBurn` underflows and reverts with panic 0x11. There is a remainder whenever the price is not exactly 1:1 (`calculateCapitalAccount() != totalSupply()`, true after any realised interest, gain, or loss), i.e. the normal state of a live pool.

For example, with `totalSupply = 1,000,000`, `capitalAccount = 1,000,199`, and free `_totalAssets = 43,532`: $43,532 * 1,000,000 / 1,000,199 = 43,523.34\dots$, so `maxRedeemableShares = 43,523` (floor) but `sharesToBurn = 43,524` (ceil), and $43,523 - 43,524$ underflows.

Because `processRedemptionQueue` is called unconditionally by `deposit`, `reconcileSingleInvoice`, `impairInvoice`, and `unfactorInvoice`, a single such entry at the queue head reverts all of these operations. New deposits are blocked, paid invoices cannot be reconciled (LP gains and invoice NFTs stuck), and impairments/unfactors revert. The queue cannot drain itself because processing reverts; recovery requires the owner to `cancelQueuedRedemption` (compacts the entry) or the pool owner to swap the queue via `setRedemptionQueue`. This is reachable by any permitted redeemer during a normal illiquid period, and can be induced deliberately as a grieving vector.

Proof of concept: The steps are:

1. Alice deposits 1,000,000; a small invoice is funded and repaid, realising interest, so `capitalAccount != totalSupply` (price off 1:1, with a remainder).
2. A large invoice (1,200,000, upfront 80%) is funded, dropping free `totalAssets` to a small amount.
3. Alice withdraws her whole position; the bulk queues as an asset-based entry larger than the free liquidity.
4. An unrelated user deposits 3,333. This triggers `processRedemptionQueue`, where `assetsToWithdraw = min(queued, totalAssets) = totalAssets` (full drain), `sharesToBurn` (ceil) exceeds `maxRedeemableShares` (floor), and the deposit reverts with panic 0x11 (arithmetic underflow).

The test below extends the team's existing `CommonSetup` harness and uses only existing helpers, so it drops straight into `test/foundry/`. Full file also at `leads/poc/TestAuditQueueUnderflow.t.sol`.

Run with:

```
FOUNDRY_PROFILE=test forge test --match-contract TestAuditQueueUnderflow -vv
```

(The test profile is required: the default profile hits a solc "Tag too large" internal error at `optimizer_runs=200`; the test profile uses `optimizer_runs=10`.)

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./CommonSetup.t.sol";

/// @notice PoC for the redemption-queue processing underflow DoS (Lead 02).
/// @dev Drop into test/foundry/ and run:
///      FOUNDRY_PROFILE=test forge test --match-contract TestAuditQueueUnderflow -vv
contract TestAuditQueueUnderflow is CommonSetup {
    function testQueueProcessingWithdrawUnderflowDoS() public {
        // 1. Alice deposits.
        vm.prank(alice);
        bullaFactoring.deposit(1_000_000, alice);

        // 2. Fund and fully repay a small invoice to realise interest, so the share price
        //     is no longer exactly 1:1 (capitalAccount != totalSupply, with a remainder).
        vm.prank(bob);
        uint256 inv1 = createClaim(bob, charlie, 100_000, dueBy);
        vm.prank(underwriter);
        _approveInvoice(inv1, interestApr, spreadBps, upfrontBps, 0);
        vm.startPrank(bob);
        bullaClaim.approve(address(bullaFactoring), inv1);
        _fundInvoice(inv1, upfrontBps, address(0));
        vm.stopPrank();
        vm.warp(block.timestamp + 10 days);
        vm.startPrank(charlie);
        asset.approve(address(bullaClaim), 100_000);
        bullaClaim.payClaim(inv1, 100_000);
        vm.stopPrank();
        assertTrue(
            bullaFactoring.calculateCapitalAccount() != bullaFactoring.totalSupply(),
            "price should be off 1:1"
        );

        // 3. Lock almost all liquidity in a large invoice so free totalAssets is small.
        vm.prank(bob);
        uint256 inv2 = createClaim(bob, charlie, 1_200_000, block.timestamp + 30 days);
        vm.prank(underwriter);
        _approveInvoice(inv2, interestApr, spreadBps, upfrontBps, 0);
        vm.startPrank(bob);
        bullaClaim.approve(address(bullaFactoring), inv2);
        _fundInvoice(inv2, upfrontBps, address(0));
        vm.stopPrank();

        // 4. Alice withdraws her whole position -> the bulk queues as an asset-based entry
        //     that is far larger than the free liquidity.
        uint256 aliceMaxAssets = bullaFactoring.previewRedeem(bullaFactoring.balanceOf(alice));
        vm.prank(alice);
        bullaFactoring.withdraw(aliceMaxAssets, alice, alice);
        (, uint256 qAssets) = bullaFactoring.getRedemptionQueue().getTotalQueuedForOwner(alice);
        assertGt(qAssets, bullaFactoring.totalAssets(), "queued > free liquidity, so processing fully
        ↪ drains");

        // 5. Any deposit now triggers processRedemptionQueue. assetsToWithdraw clamps to the
        //     free liquidity (a full drain), so sharesToBurn = previewWithdraw(...) [ceil]
        //     exceeds maxRedeemableShares = convertToShares(...) [floor], and
        //     `maxRedeemableShares -= sharesToBurn` (1166) underflows -> the deposit reverts.
        permitUser(address(0xBEEF), false, 5000);
        vm.prank(address(0xBEEF));
        vm.expectRevert(stdError.arithmeticError); // panic 0x11 (arithmetic underflow)
        bullaFactoring.deposit(3333, address(0xBEEF));
    }
}

```

```
}  
}
```

Recommendation: Do not maintain `maxRedeemableShares` as an independent running tally that can diverge from the live conversions. Either clamp the subtraction (`maxRedeemableShares = sharesToBurn > maxRedeemableShares ? 0 : maxRedeemableShares - sharesToBurn;`), and likewise the share branch at [L1142](#)), or gate the payout on `_totalAssets` alone and recompute the share cap live each iteration. Add a regression test that queues an asset-based withdrawal draining all liquidity at a non-1:1 price.

Bulla: Resolved in [PR 243](#).

LPHAC: Verified fix. `processRedemptionQueue` now clamps the subtraction (`maxRedeemableShares = sharesToBurn >= maxRedeemableShares ? 0 : maxRedeemableShares - sharesToBurn`) and a regression test (`testDepositDoesNotRevertWhenQueueDrainsAllLiquidity`) was added. Confirmed on main (`cba5cb4`): the previously-reverting deposit now succeeds, and the solvency invariant holds across ~24,000 fuzzed operations.

7.3 Medium

7.3.1 `unfactorInvoice` does not reclaim `insurancePremium` causing insurance balance solvency deficit

Severity: Medium

Context: [BullaFactoring.sol#L661-L678](#), [BullaFactoring.sol#L687-L731](#), [FeeCalculations.sol#L159-L193](#)

Description: At funding time, both `protocolFee` and `insurancePremium` are withheld from `fundedAmountGross` by `FeeCalculations.calculateTargetFees` and credited into `protocolFeeBalance` and `insuranceBalance` respectively. These two balances are conceptually symmetric - they each represent a claim by a specific party (DAO and insurer) that is backed by pool cash retained at funding.

`_calculateUnfactorAmounts` charges the unfactoring party to make the pool whole, but it only re-adds `protocolFee` and **omits** `insurancePremium`:

```
// BullaFactoring.sol:677
totalRefundOrPaymentAmount = int256(
    approval.fundedAmountNet
    + trueInterest + trueSpreadAmount + trueAdminFee
    + ApprovalPacking.protocolFee(approval)
) - int256(paymentSinceFunding);
```

The rest of `unfactorInvoice` never touches `insuranceBalance` either. So after an unfactor:

- `insuranceBalance` still has the original `insurancePremium` credited at funding.
- The pool's actual token balance is short by exactly `insurancePremium` relative to the sum of all outstanding withdrawable claims.

Repeated unfactors compound the deficit. The contract has no on-chain mechanism to detect or correct it, and pool owner cannot top up `insuranceBalance` without violating the conservation invariant elsewhere.

Recommendation: Add `insurancePremium` to the unfactor settlement amount, treating it symmetrically with `protocolFee`:

```
totalRefundOrPaymentAmount = int256(
    approval.fundedAmountNet
    + trueInterest + trueSpreadAmount + trueAdminFee
-   + ApprovalPacking.protocolFee(approval)
+   + ApprovalPacking.protocolFee(approval)
+   + ApprovalPacking.insurancePremium(approval)
) - int256(paymentSinceFunding);
```

If the intent is that insurance premium is non-refundable from the factorer's perspective ("the pool was insured during the time it held the invoice"), then the symmetric fix is to decrement `insuranceBalance` by `insurancePremium` inside `unfactorInvoice` at the same time `_activeInvoices` is cleared — this drops the insurer's claim back in line with pool cash. Either path closes the invariant; the current code does neither.

Bulla: Resolved in [PR 232](#).

LPHAC: Verified fix. `_calculateUnfactorAmounts` now re-adds `insurancePremium` to `totalRefundOrPaymentAmount` alongside `protocolFee`, restoring the invariant that pool cash backs every outstanding withdrawable claim after an unfactor.

7.3.2 `deposit` only validates the caller, so compliant users can mint pool shares to non-compliant recipients

Context: [BullaFactoring.sol#L203-L211](#), [BullaFactoring.sol#L381-L385](#)

Description: The new `_update()` restriction only checks `depositPermissions` for non-mint and non-burn transfers. The comment assumes minting is "already gated by `deposit()`", but `deposit()` only checks `depositPermissions.isAllowed(_msgSender())` and does not validate the receiver. `super.deposit()` then mints shares directly to receiver.

A KYC-approved address can deposit assets and nominate an arbitrary receiver that would fail the sanctions/KYC/agreement checks. Because minting is exempt inside `_update()`, the disallowed receiver receives pool shares even though direct transfers to that same address would revert. This bypass undermines the new compliance model for secondary share ownership and contradicts the contract's own assumption that minting is already protected by the deposit flow.

Proof of concept scenario:

1. Address A passes compliance and is allowed by `depositPermissions`.
2. Address B is not allowed, for example because it is not KYC'd or has not signed the current agreement.
3. A calls `deposit/assets, B)`.
4. The call succeeds because `deposit()` only checks A, and `_update()` skips permission checks when `from == address(0)`.
5. B ends up holding pool shares even though `transfer(B, ...)` and `transfer(..., B)` were meant to be blocked.

Recommendation: Validate receiver in `deposit()` before minting shares. If the design intends every pool-token holder to satisfy `depositPermissions`, require `depositPermissions.isAllowed(receiver)` alongside the existing caller check. Keeping the invariant in one place by enforcing recipient checks on mints inside `_update()` would also reduce the chance of similar bypasses if another minting path is introduced later.

Bulla: Resolved in [PR 237](#).

LPHAC: Verified fix. `deposit` now reverts with `UnauthorizedDeposit(receiver)` when the receiver is not allowed by `depositPermissions`, restoring the invariant that every share holder satisfies the compliance gate. The fix is scoped to `deposit` rather than the more general `_update` mint path, so any future minting entry point will need to repeat the check.

7.3.3 Factory-created pools permanently lose `RedemptionQueue` admin control

Context: [BullaFactoring.sol#L175-L194](#), [BullaFactoringFactoryV2_1.sol#L176-L209](#), [RedemptionQueue.sol#L47-L56](#)

Description: `BullaFactoringV2_2` constructs its `RedemptionQueue` with `msg.sender` as the queue owner. When the pool is deployed through `BullaFactoringFactoryV2_1`, `msg.sender` inside the pool constructor is the factory contract, not the eventual pool creator. The factory later transfers pool ownership, but it never transfers `RedemptionQueue` ownership.

Every pool created via the factory ends up with a `RedemptionQueue` whose owner is the factory contract address. Since the factory contract exposes no forwarding methods for queue administration, owner-only queue operations become effectively inaccessible:

- `setMaxQueueSize()`
- `clearQueue()`
- `compactQueue()`
- `setFactoringContract()`

This is not merely cosmetic. If the queue approaches the default `maxQueueSize` of 500, or if operators need to clear or compact it during incident response, the pool creator has no authority to do so. Because queued redemptions are core to illiquid-exit handling, this creates a durable operations and availability risk for every factory-deployed pool.

Proof of concept scenario:

1. A user creates a pool through `BullaFactoringFactoryV2_1.createPool(...)`.
2. The pool is deployed and ownership is transferred to the pool creator.
3. The creator attempts to call `pool.getRedemptionQueue().setMaxQueueSize(...)`.
4. The call reverts because `Ownable(owner)` on the queue was set to the factory contract, not the pool creator.
5. No externally owned account can recover this control because the queue owner is a contract with no admin-forwarding surface.

Recommendation: Pass the intended pool owner into the pool constructor and use that address for `RedemptionQueue` ownership, or transfer queue ownership immediately after deployment before returning from the factory. The pool and queue ownership model should be aligned so that the same operator can manage queue-related safety controls.

Bulla: Acknowledged. The factory contract is out of scope for this audit and is not currently used.

LPHAC: Acknowledged as out of scope. Finding retained for visibility.

7.3.4 Impairment `principalLoss` clamp discards positive surplus, permanently stranding pool funds

Context: [BullaFactoring.sol#L1001-L1005](#)

Description: When the insurer impairs an invoice, `impairInvoice` measures the LP principal loss against `fundedAmountGross`, clamps it at zero, and discards the positive side:

```
// BullaFactoring.sol:1001-1005
uint256 credited = lpCredit + paymentsSinceFunding;
uint256 _principalLoss = _approval.fundedAmountGross > credited
    ? _approval.fundedAmountGross - credited
    : 0;
// <- when credited > fundedAmountGross the surplus is dropped
impairmentLosses += _principalLoss;
```

`credited = lpCredit + paymentsSinceFunding` where `lpCredit = impairmentGrossGain + poolOwnedWithheld - feesCharged`. When `credited > fundedAmountGross` the invoice was net-profitable at impairment, but the surplus `credited - fundedAmountGross` is never credited to `paidInvoicesGain`. The pool holds that cash but no claim accounts for it, so `pricePerShare` does not rise and the surplus is **permanently stranded**: it survives a full wind-down with `totalSupply == 0`, leaving USDC locked in the contract that no one can withdraw.

This is reachable with **default insurance params**. Two triggers:

- A mostly-paid invoice that is then impaired: fund 100k at 80% (gross 80k), debtor pays 90k, insurer impairs. `paymentsSinceFunding = 90k > fundedAmountGross`, so 10,000 is stranded.
- A high `impairmentGrossGainBps` (up to 100% is allowed): the large insurance payout pushes `credited` above gross; 18,887 stranded in the PoC.

Proof of concept: `leads/poc/TestImpairmentSurplusRegression.t.sol`. It asserts the CORRECT (post-fix) behaviour: after a full wind-down the pool is exactly empty. All three cases FAIL on the current clamp and PASS with the fix below.

```
FOUNDRY_PROFILE=test forge test --match-contract TestImpairmentSurplusRegression -vv
```

Full drop-in test (extends the existing `CommonSetup` harness, uses only existing helpers):

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```

import "./CommonSetup.t.sol";

/// @notice Regression test for the impairment principalLoss clamp discarding positive surplus.
/// When `credited` (lpCredit + paymentsSinceFunding) exceeds fundedAmountGross at impairment,
/// the surplus must be recognised as an LP gain, not silently dropped. Otherwise pool cash
/// exceeds the sum of claims and the excess is permanently stranded (unredeemable, and locked
/// forever once totalSupply hits 0).
///
/// These tests assert the CORRECT (post-fix) behaviour: after a full wind-down the pool is
/// exactly empty. They FAIL on the buggy clamp and PASS once the surplus is credited.
contract TestImpairmentSurplusRegression is CommonSetup {
    address insurerAddr = address(0x1999);

    function _windDownAndAssertEmpty(string memory tag) internal {
        if (bullaFactoring.insuranceBalance() > 0) { vm.prank(insurerAddr);
            ↪ bullaFactoring.withdrawInsuranceBalance(); }
        uint256 shares = bullaFactoring.maxRedeem(alice);
        if (shares > 0) { vm.prank(alice); bullaFactoring.redeem(shares, alice, alice); }
        if (bullaFactoring.protocolFeeBalance() > 0) bullaFactoring.withdrawProtocolFees();
        if (bullaFactoring.adminFeeBalance() > 0) { vm.prank(bullaFactoring.owner());
            ↪ bullaFactoring.withdrawAdminFeesAndSpreadGains(); }
        assertEq(bullaFactoring.balanceOf(alice), 0, string.concat(tag, ": alice fully redeemed"));
        assertEq(bullaFactoring.calculateCapitalAccount(), 0, string.concat(tag, ": capital account
            ↪ zero"));
        assertEq(asset.balanceOf(address(bullaFactoring)), 0, string.concat(tag, ": no stranded cash"));
    }

    // Default insurance params. Invoice paid 90k (> ~80k gross) then impaired -> surplus.
    function testMostlyPaidThenImpairedCreditsSurplus() public {
        asset.mint(insurerAddr, 1_000_000); vm.prank(insurerAddr);
        ↪ asset.approve(address(bullaFactoring), type(uint256).max);
        vm.prank(alice); bullaFactoring.deposit(1_000_000, alice);
        vm.prank(bob); uint256 id = createClaim(bob, charlie, 100_000, dueBy);
        vm.prank(underwriter); _approveInvoice(id, interestApr, spreadBps, upfrontBps, 0);
        vm.startPrank(bob); bullaClaim.approve(address(bullaFactoring), id); _fundInvoice(id,
            ↪ upfrontBps, address(0)); vm.stopPrank();
        vm.startPrank(charlie); asset.approve(address(bullaClaim), 90_000); bullaClaim.payClaim(id,
            ↪ 90_000); vm.stopPrank();
        vm.warp(block.timestamp + 91 days);
        vm.prank(insurerAddr); bullaFactoring.impairInvoice(id);
        _windDownAndAssertEmpty("mostly-paid");
    }

    // High impairmentGrossGainBps (100%), impaired unpaid then fully recovered.
    function testHighGrossGainImpairThenRecoverStaysClean() public {
        asset.mint(insurerAddr, 10_000_000); vm.prank(insurerAddr);
        ↪ asset.approve(address(bullaFactoring), type(uint256).max);
        bullaFactoring.setInsuranceParams(uint16(100), uint16(10000), uint16(5000));
        vm.prank(alice); bullaFactoring.deposit(1_000_000, alice);
        vm.prank(bob); uint256 id = createClaim(bob, charlie, 100_000, dueBy);
        vm.prank(underwriter); _approveInvoice(id, interestApr, spreadBps, upfrontBps, 0);
        vm.startPrank(bob); bullaClaim.approve(address(bullaFactoring), id); _fundInvoice(id,
            ↪ upfrontBps, address(0)); vm.stopPrank();
        vm.warp(block.timestamp + 91 days);
        vm.prank(insurerAddr); bullaFactoring.impairInvoice(id);
        // full recovery
        vm.startPrank(charlie); asset.approve(address(bullaClaim), 100_000); bullaClaim.payClaim(id,
            ↪ 100_000); vm.stopPrank();
        _windDownAndAssertEmpty("high-grossgain-recover");
    }

    // High impairmentGrossGainBps (100%), impaired unpaid, never recovers.

```

```

function testHighGrossGainImpairNoRecoveryStaysClean() public {
    asset.mint(insurerAddr, 10_000_000); vm.prank(insurerAddr);
    ↪ asset.approve(address(bullaFactoring), type(uint256).max);
    bullaFactoring.setInsuranceParams(uint16(100), uint16(10000), uint16(5000));
    vm.prank(alice); bullaFactoring.deposit(1_000_000, alice);
    vm.prank(bob); uint256 id = createClaim(bob, charlie, 100_000, dueBy);
    vm.prank(underwriter); _approveInvoice(id, interestApr, spreadBps, upfrontBps, 0);
    vm.startPrank(bob); bullaClaim.approve(address(bullaFactoring), id); _fundInvoice(id,
    ↪ upfrontBps, address(0)); vm.stopPrank();
    vm.warp(block.timestamp + 91 days);
    vm.prank(insurerAddr); bullaFactoring.impairInvoice(id);
    _windDownAndAssertEmpty("high-grossgain-norecover");
}
}

```

Recommendation (proposed fix, verified): recognise the surplus as an LP gain instead of dropping it, symmetric with how a shortfall debits impairmentLosses:

```

uint256 credited = lpCredit + paymentsSinceFunding;
uint256 _principalLoss;
if (_approval.fundedAmountGross > credited) {
    _principalLoss = _approval.fundedAmountGross - credited;
    impairmentLosses += _principalLoss;
} else {
    // credited exceeds the gross funded exposure: the invoice was net-profitable at
    // impairment. Recognise the surplus as an LP gain instead of stranding it.
    paidInvoicesGain += credited - _approval.fundedAmountGross;
}

```

Bulla: Resolved in [PR 245](#).

LPHAC: Verified fix. When credited > fundedAmountGross, impairInvoice now credits the surplus to paidInvoicesGain (paidInvoicesGain += credited - fundedAmountGross) instead of dropping it, so the previously-stranded profit accrues to LPs and the pool winds down to exactly zero.

7.3.5 Partial payments on an impaired invoice are never credited and become permanently stranded

Context: [BullaFactoring.sol#L612-L648](#), [BullaFactoring.sol#L972-L975](#)

Description: impairInvoice removes the invoice from _activeInvoices, and recovery is handled only by the impaired branch of reconcileSingleInvoice, which requires the invoice to be fully paid:

```

// BullaFactoring.sol:612-620
function reconcileSingleInvoice(uint256 invoiceId) external {
    IInvoiceProviderAdapterV2.Invoice memory invoice =
    ↪ invoiceProviderAdapter.getInvoiceDetails(invoiceId);
    if (!invoice.isPaid) revert InvoiceNotPaid(); // <- recovery only fires on FULL payment
    ...
    uint256 recoveredAmount = invoice.paidAmount - _impairment.paidAmountAtImpairment;
}

```

A debtor who pays an impaired invoice **partially** (but never in full) sends that cash to the pool (which owns the invoice NFT), but no reconcile ever fires, so the partial recovery is never credited to insuranceBalance or paidInvoicesGain. The cash sits unattributed: pricePerShare does not reflect it, LPs cannot redeem it, and once the pool is wound down (totalSupply == 0) it is permanently locked.

This is a realistic scenario: a distressed debtor on a written-off invoice pays some of what they owe but not all. The full-payment gate means anything short of 100% is stranded.

Proof of concept: leads/poc/TestAuditPartialRecovery.t.sol. Fund a 100k invoice, impair it, debtor pays 50k (partial, no reconcile), then wind down every party. Result: 50,000 permanently stranded with totalSupply == 0.

```
FOUNDRY_PROFILE=test forge test --match-contract TestAuditPartialRecovery -vv
```

Full drop-in test (extends the existing CommonSetup harness, uses only existing helpers):

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "./CommonSetup.t.sol";

/// @notice An impaired invoice is removed from _activeInvoices, and the recovery branch only fires
/// on FULL payment (reconcileSingleInvoice requires isPaid). A PARTIAL payment after impairment
/// therefore lands in the pool as cash but is never credited to anyone. If full payment never
/// comes, that partial recovery is stranded (permanently locked once totalSupply hits 0).
contract TestAuditPartialRecovery is CommonSetup {
    address insurerAddr = address(0x1999);
    function _claims() internal view returns (uint256) {
        return bullaFactoring.calculateCapitalAccount() + bullaFactoring.protocolFeeBalance()
            + bullaFactoring.adminFeeBalance() + bullaFactoring.insuranceBalance();
    }
    function testPartialPaymentAfterImpairmentIsStranded() public {
        asset.mint(insurerAddr, 1_000_000); vm.prank(insurerAddr);
        ↪ asset.approve(address(bullaFactoring), type(uint256).max);
        vm.prank(alice); bullaFactoring.deposit(1_000_000, alice);
        vm.prank(bob); uint256 id = createClaim(bob, charlie, 100_000, dueBy);
        vm.prank(underwriter); _approveInvoice(id, interestApr, spreadBps, upfrontBps, 0);
        vm.startPrank(bob); bullaClaim.approve(address(bullaFactoring), id); _fundInvoice(id,
        ↪ upfrontBps, address(0)); vm.stopPrank();

        vm.warp(block.timestamp + 91 days);
        vm.prank(insurerAddr); bullaFactoring.impairInvoice(id);
        assertEq(bullaFactoring.getActiveInvoicesCount(), 0, "impaired invoice removed from active");

        // Debtor pays 50k PARTIAL (not full) -> no reconcile callback fires.
        vm.startPrank(charlie); asset.approve(address(bullaClaim), 50_000); bullaClaim.payClaim(id,
        ↪ 50_000); vm.stopPrank();

        uint256 cash = asset.balanceOf(address(bullaFactoring));
        uint256 claims = _claims();
        emit log_named_uint("pool cash", cash);
        emit log_named_uint("claims", claims);
        if (cash > claims) emit log_named_uint("STRANDED (cash - claims)", cash - claims);

        // Wind down everyone (invoice never fully pays).
        if (bullaFactoring.insuranceBalance() > 0) { vm.prank(insurerAddr);
        ↪ bullaFactoring.withdrawInsuranceBalance(); }
        uint256 shares = bullaFactoring.maxRedeem(alice);
        if (shares > 0) { vm.prank(alice); bullaFactoring.redeem(shares, alice, alice); }
        if (bullaFactoring.protocolFeeBalance() > 0) bullaFactoring.withdrawProtocolFees();
        if (bullaFactoring.adminFeeBalance() > 0) { vm.prank(bullaFactoring.owner());
        ↪ bullaFactoring.withdrawAdminFeesAndSpreadGains(); }
        emit log_named_uint("FINAL pool balance (permanently stranded, totalSupply 0)",
        ↪ asset.balanceOf(address(bullaFactoring)));
        emit log_named_uint("FINAL totalSupply", bullaFactoring.totalSupply());
    }
}
```

Recommendation: Allow impaired invoices to recognise partial recoveries rather than gating on full payment. Options: let `reconcileSingleInvoice` process the incremental amount received since the last reconcile for an impaired invoice (tracking a running `recoveredSoFar`), or provide an insurer/permissionless "recover partial" endpoint that credits `invoice.paidAmount - paidAmountAtImpairment` and advances `paidAmountAtImpairment`. Whatever the mechanism, the invariant to hold is that cash received on an impaired invoice is always attributable

to a claim (insurer / LP), not stranded.

Bulla: Acknowledged as a known drawback: partial payments on an impaired invoice are only credited once it is fully repaid.

LPHAC: Acknowledged, retained for visibility.

7.4 Low

7.4.1 `impairmentGrossGainBps` is read live in `previewImpair`, allowing retroactive change of insurance economics for already-funded invoices

Severity: Low

Context: [BullaFactoring.sol#L934-L950](#), [BullaFactoring.sol#L916-L925](#)

Description: At funding time, `insurancePremium` is computed using the then-current `insuranceFeeBps` and snapshotted into `approval.protocolFeeAndInsurancePremium` (via `ApprovalPacking.packFees`). This correctly locks the premium charged to the factorer.

The corresponding payout side, however, is read live. `previewImpair` computes the insurer's `impairmentGrossGain` payout using the current `impairmentGrossGainBps` storage value:

```
// BullaFactoring.sol:946
impairmentGrossGain = Math.mulDiv(outstandingBalance, impairmentGrossGainBps, 10000);
```

`impairmentGrossGainBps` is mutable by the pool owner via `setInsuranceParams` at any time, including after invoices have been funded:

```
// BullaFactoring.sol:916-925
function setInsuranceParams(uint16 _insuranceFeeBps, uint16 _impairmentGrossGainBps, uint16
↳ _recoveryProfitRatioBps) external onlyOwner {
    ...
    impairmentGrossGainBps = _impairmentGrossGainBps;
    ...
}
```

This produces an economic asymmetry. The factorer's premium is locked at funding, but the insurer's payout ratio against that premium can be changed unilaterally by the pool owner up until the moment of `impairment.recoveryProfitRatioBps` (used in the recovery branch of `reconcileSingleInvoice` at line 611) is also read live and has the same property, controlling how recovered cash on a previously impaired invoice is split between insurer and investors.

To be clear, the insurer signs up under one set of parameters with a specific risk model, a certain `insurancePremium:impairmentGrossGain` ratio which allows the insurer to accurately assess profitability dependent on probability of impairment. However, this dynamic completely changes if the payout is based on live values, rather than values at funding.

Recommendation: Either snapshot `impairmentGrossGainBps` (and likely `recoveryProfitRatioBps`) into the `InvoiceApproval` at funding time, alongside `insurancePremium`, so all insurance economics for a given invoice are fixed once funded; or document this as an explicit insurer trust assumption and ensure the off-chain agreement with insurers reflects that the pool owner can change payout ratios for in-flight invoices.

Bulla: Resolved in [PR 233](#).

LPHAC: Verified fix. Both `impairmentGrossGainBps` and `recoveryProfitRatioBps` are now snapshotted into `InvoiceApproval.feeParams` at funding, and `previewImpair` and the recovery branch of `reconcileSingleInvoice` read from the snapshot. Owner changes via `setInsuranceParams` no longer affect insurer payouts or recovery splits on already-funded invoices.

7.4.2 Impairment released the withheld-fee wedge without writing it down, leaving the pool insolvent by

`fundedAmountGross - fundedAmountNet`

Context: [BullaFactoring.sol#L458](#), [BullaFactoring.sol#L473](#), [BullaFactoring.sol#L778-L781](#), [BullaFactoring.sol#L1002-L1005](#)

Description: At funding the pool pays out only `fundedAmountNet`, but books the full `fundedAmountGross` as at-risk and tracks the difference (the wedge) in `withheldFees`:

```
// BullaFactoring.sol:458 / 473
totals[3] += fundedAmountGross - fundedAmountNet; // the wedge (protocol fee + insurance premium +
↳ withheld interest/spread/admin)
...
withheldFees += totals[3];
```

The wedge is expected fee income that is only real once the debtor pays. On a normal payment the debtor's cash backs it. On impairment, `removeActivePaidInvoice` releases the full gross and the wedge from the trackers, exactly as it would for a genuinely-paid invoice:

```
// BullaFactoring.sol:778 / 781
capitalAtRiskPlusWithheldFees -= atRiskPlusWithheldFees; // releases full
↳ gross
withheldFees -= atRiskPlusWithheldFees - approvedInvoices[invoiceId].fundedAmountNet; // releases the
↳ wedge
```

But the only offsetting entry `impairInvoice` booked was a principal loss measured against `fundedAmountNet`, which never covers the wedge. The withheld cash was therefore counted twice: once as LP capital that was never reduced, and once as a loss offset, while the protocol/admin fee claims backed by that same cash stayed outstanding. Concretely (100k invoice, 80% advance): after `impair` + full recovery + LP exit the pool held 113 in cash against 2,804 of fee claims, so whoever withdrew fees last hit an ERC20 revert, short by exactly the 2,691 wedge.

The derivation is exact: `deficit = fundedAmountGross - fundedAmountNet`. Substituting the loss formula and the recovery relations, every other term (the insurance payout, the admin fee `feesCharged`, the payments-since-funding) cancels, leaving only the wedge.

Recommendation (implemented): Measure the loss against `fundedAmountGross` instead of `fundedAmountNet`, so the withheld-fee carve-outs are charged to LPs when the invoice is written off:

```
// BullaFactoring.sol:1002 (post-fix)
uint256 _principalLoss = _approval.fundedAmountGross > credited
? _approval.fundedAmountGross - credited
: 0;
```

The recovery branch needs no change (`insuranceShare + investorShare == recoveredAmount` is internally balanced). PR 244 also added `testImpairThenRecoveryKeepsPoolSolvent` and `testFullRedemptionAfterImpairmentBeforeRecoveryIsSolvent`, and updated the two loss tests from the net to the gross base (each was previously off by exactly the wedge).

Bulla: Resolved in [PR 244](#).

LPHAC: Verified fix at commit `cba5cb4`.

7.4.3 Funding an invoice with $\text{upfrontBps} < \text{protocolFeeBps} + \text{insuranceFeeBps}$ makes the pool instantly insolvent

Context: [FeeCalculations.sol#L175-L190](#), [BullaFactoring.sol#L466-L472](#)

Description: The protocol fee and insurance premium are computed on the **full** `initialInvoiceValue`, but only `fundedAmountGross` (based on the upfront) is reserved as capital-at-risk, and `netFundedAmount` clamps to 0:

```
// FeeCalculations.sol:175-190
protocolFee      = Math.mulDiv(approval.initialInvoiceValue, protocolFeeBps, 10000); // full value
insurancePremium = Math.mulDiv(approval.initialInvoiceValue, insuranceFeeBps, 10000); // full value
fundedAmountGross = Math.mulDiv(approval.initialInvoiceValue, factorerUpfrontBps, 10000); // upfront only
...
netFundedAmount = fundedAmountGross > totalFees ? fundedAmountGross - totalFees : 0;
```

At funding, `protocolFeeBalance` and `insuranceBalance` are credited these full-value fees, while `capitalAtRiskPlusWithheldFees` is increased by only `fundedAmountGross`:

```
// BullaFactoring.sol:466-472
if (totals[2] > _totalAssets) revert InsufficientFunds(...); // only checks fundedAmountGross
...
protocolFeeBalance += totals[0]; // full-value protocol fee
insuranceBalance   += totals[1]; // full-value insurance premium
capitalAtRiskPlusWithheldFees += totals[2]; // only fundedAmountGross reserved
```

When $\text{upfrontBps} < \text{protocolFeeBps} + \text{insuranceFeeBps}$, the fee claims credited exceed the reserved capital, and the pool is insolvent the instant the invoice is funded: $\text{poolCash} < \text{totalAssets} + \text{protocolFeeBalance} + \text{insuranceBalance}$ by $(\text{protocolFee} + \text{insurancePremium}) - \text{fundedAmountGross}$. The DAO and insurer can then withdraw the over-booked fees and leave LPs unable to redeem in full. The liquidity check only compares `fundedAmountGross`, so it does not catch the over-booking.

All three values are in basis points. With default params ($\text{protocolFeeBps}=25$, $\text{insuranceFeeBps}=100$, i.e. 0.25% and 1%) the threshold $\text{protocolFeeBps} + \text{insuranceFeeBps}$ is 125 bps, so the pool is only insolvent when the upfront is under 1.25%, which is unusual. But insuranceFeeBps can be set as high as 10000 (100%): a pool that charges a 10% insurance fee ($\text{insuranceFeeBps}=1000$) is insolvent for any upfront under ~10.25% ($1000 + 25$ bps), which is plausible for high-risk factoring.

Proof of concept: `leads/poc/TestAuditLowUpfront.t.sol` (deposit 1M, fund a 400k invoice at 1% upfront under default params) shows the pool insolvent by 1,000 immediately after funding, and the LP short after the DAO/insurer withdraw.

```
FOUNDRY_PROFILE=test forge test --match-contract TestAuditLowUpfront -vv
```

Full drop-in test (extends the existing `CommonSetup` harness, uses only existing helpers):

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "./CommonSetup.t.sol";

/// @notice Funding an invoice with  $\text{upfrontBps} < \text{protocolFeeBps} + \text{insuranceFeeBps}$  makes the pool
/// instantly insolvent, because protocolFee and insurancePremium are charged on the full
/// initialInvoiceValue while only the (smaller) fundedAmountGross is reserved. Asserts the bug is
/// present (this PASSES on unfixed main and should FAIL once a guard/cap is added).
contract TestAuditLowUpfront is CommonSetup {
    function _obligations() internal view returns (uint256) {
        return bullaFactoring.totalAssets() + bullaFactoring.protocolFeeBalance()
            + bullaFactoring.adminFeeBalance() + bullaFactoring.insuranceBalance();
    }

    function testLowUpfrontFundingIsInstantlyInsolvent() public {
        // default params:  $\text{protocolFeeBps}=25$ ,  $\text{insuranceFeeBps}=100 \Rightarrow$  threshold ~125 bps
        vm.prank(alice); bullaFactoring.deposit(1_000_000, alice);
        uint16 lowUpfront = 100; // 1% < 1.25% threshold
    }
}
```

```

vm.prank(bob); uint256 id = createClaim(bob, charlie, 400_000, dueBy);
vm.prank(underwriter); _approveInvoice(id, interestApr, spreadBps, lowUpfront, 0);
vm.startPrank(bob); bullaClaim.approve(address(bullaFactoring), id); _fundInvoice(id,
↳ lowUpfront, address(0)); vm.stopPrank();

uint256 cash = asset.balanceOf(address(bullaFactoring));
emit log_named_uint("pool cash", cash);
emit log_named_uint("obligations (totalAssets+fees)", _obligations());
if (_obligations() > cash) emit log_named_uint("INSOLVENT by", _obligations() - cash);
assertLt(cash, _obligations(), "pool should be insolvent right after low-upfront funding");
}
}

```

Recommendation: Enforce the invariant that fees cannot exceed what is funded. Simplest: require `upfrontBps` $\geq$ `protocolFeeBps` + `insuranceFeeBps` at approval/funding (revert otherwise). Alternatively, cap `protocolFee` + `insurancePremium` at `fundedAmountGross`, or charge the insurance premium on the funded amount rather than the full invoice value. If a setter-level guard is preferred (as discussed), reject `setInsuranceParams` / `setProtocolFeeBps` values whose sum could exceed the minimum permitted upfront; note that upfront is chosen per-invoice down to 1 bps, so a setter guard alone must be paired with a minimum-upfront check at approval to fully close it.

Bulla: Resolved in [PR 246](#).

LPHAC: Verified fix. `_fundInvoice` now reverts with `FeesExceedFundedAmount` when `protocolFee` + `insurancePremium` $\geq$ `fundedAmountGross` (equivalently `upfrontBps` $\leq$ `protocolFeeBps` + `insuranceFeeBps`), so an invoice can no longer be funded into an instantly-insolvent state.

7.5 Informational

7.5.1 `viewPoolStatus` pagination assumes stable iteration order which `EnumerableSet` does not provide

Severity: Informational

Context: [BullaFactoring.sol#L552-L585](#)

Description: `viewPoolStatus(offset, limit)` is documented for use by automation services and paginates by iterating `_activeInvoices.at(i)` from `offset` to `offset + limit`. `_activeInvoices` is an `EnumerableSet.UintSet` whose `remove` swaps the removed element with the last and pops, so positions change whenever an invoice is reconciled, impaired, or unfactored between paginated calls. Across consecutive paginated reads, active invoices can be skipped or visited twice.

Recommendation: Surface a staleness signal alongside the paginated results so off-chain consumers can detect when the underlying active set has shifted between calls and restart their iteration. A monotonically-incrementing version that changes on every add or remove to `_activeInvoices` is sufficient; the set length alone is not, since a remove-then-add sequence in the same window preserves length while reshuffling positions.

Bulla: Resolved in [PR 234](#).

LPHAC: Verified fix. A monotonic `activeInvoicesVersion` is now bumped on every `_activeInvoices` add and remove, with `NatSpec` on `viewPoolStatus` instructing paginated callers to pin the version across pages and restart iteration when it changes.

7.5.2 Queued redemptions bypass current `redeemPermissions` after permissions are revoked

Context: [BullaFactoring.sol#L1033-L1092](#), [BullaFactoring.sol#L1099-L1163](#)

Description: `redeem()` and `withdraw()` check `redeemPermissions` only when the queue entry is created. Later, `processRedemptionQueue()` processes queued exits directly through `_withdraw(...)` and does not re-check `redeemPermissions` for either the queued owner or the receiver.

A user who was permitted to request redemption at time T1 can continue to receive queued redemption proceeds even after their `redeemPermissions` are revoked at time T2. The normal direct exit path correctly blocks them after revocation, but the queued path silently bypasses that current-state permission check.

This creates a semantics mismatch: the contract presents `redeemPermissions` as the gate for redemption and withdrawal, but queued redemptions are effectively grandfathered. If the permission contract is used for emergency freezes or compliance screening, operators may incorrectly assume a revoked address can no longer exit.

Proof of concept scenario:

1. User A is currently allowed by `redeemPermissions`.
2. A requests a redemption while pool liquidity is insufficient, so most of the exit is queued.
3. An admin revokes A from `redeemPermissions`.
4. A direct `redeem()` call by A now reverts as expected.
5. Later, a deposit or invoice repayment triggers `processRedemptionQueue()`.
6. The queue drains and sends proceeds to A anyway, because `processRedemptionQueue()` bypasses the public permission checks.

Recommendation: Re-validate the queued owner (and, if intended by policy, the receiver) against `redeemPermissions` before executing each queued payout. If grandfathering is intentional, document that behavior explicitly so operators do not rely on permission revocation to halt already-queued exits.

Bulla: Resolved in [PR 239](#).

LPHAC: Verified fix. `processRedemptionQueue` now re-checks `redeemPermissions` for both the queued owner and the receiver on every payout. If either is revoked the entry is dropped without transferring assets and without burning shares, so a revoked owner retains their position and can re-queue after being re-permitted.

7.5.3 `createPool` emits and accepts permission/name metadata that it never verifies against the deployed pool

Context: [BullaFactoringFactoryV2_1.sol#L144-L220](#)

Description: The factory verifies only a subset of constructor-selected fields after deployment (`bullaDao`, `protocolFeeBps`, `invoiceProviderAdapter`, `assetAddress`). The separately supplied `poolName`, `tokenName`, `tokenSymbol`, `_depositPermissions`, `_redeemPermissions`, and `_factoringPermissions` function parameters are not checked against the actual constructor arguments embedded in `creationBytecode`.

A caller can deploy a pool whose actual permission contracts and display metadata differ from the values passed into `createPool()` and emitted in `PoolCreated`. This is especially risky because the factory is explicitly positioned as a pool-discovery surface for off-chain indexing. Indexers or UIs that trust the event payload may present incorrect permissioning or branding information for a pool that is already live and user-owned.

This is not an arbitrary-code-deployment issue. The deployed contract still uses canonical init code. The problem is that the factory's API and event surface can misrepresent unverified constructor configuration.

Proof of concept scenario:

1. The pool creator builds `creationBytecode` using permission contracts A/B/C and internal names X/Y.
2. They call `createPool()` with different non-zero permission addresses D/E/F and event strings M/N/O.
3. The factory deployment succeeds because the bytecode hash prefix and verified core parameters still match.
4. `PoolCreated` emits D/E/F and M/N/O, while the deployed pool actually uses A/B/C and X/Y.

Recommendation: Either:

- derive emitted metadata from the deployed pool itself after creation, or
- decode and verify the constructor-argument suffix of `creationBytecode` before deployment so the function inputs, event outputs, and deployed configuration cannot diverge.

Bulla: Acknowledged. The factory contract is out of scope for this audit.

LPHAC: Acknowledged as out of scope. Finding retained for visibility.